

WEBX - EXPERIMENT NO. 5

Name of Student	Anushka Shahane
Class Roll No	54
D.O.P.	06/3/25
D.O.S.	13/3/25
Sign and Grade	

AIM : To create a Flask application that demonstrates template rendering by dynamically generating HTML content using the `render_template()` function.

PROBLEM STATEMENT :

Develop a Flask application that includes:

1. A homepage route (/) displaying a welcome message with links to additional pages.
2. A dynamic route (/user/<username>) that renders an HTML template with a personalized greeting.
3. Use Jinja2 templating features, such as variables and control structures, to enhance the templates.

Theory:

1. What does the `render_template()` function do in a Flask application?

The `render_template()` function in Flask is used to render an HTML template with the data passed from the Flask application. It enables the dynamic generation of web pages by combining static HTML files (the templates) with dynamic content (variables, control structures) passed from the Python code.

Key points:

- `render_template()` is a function provided by Flask to render an HTML file located in the templates folder.
- It allows you to pass dynamic data from the Flask view function (such as variables, lists, or objects) to the HTML template.
- The function automatically uses the Jinja2 templating engine to process the HTML file, replacing placeholders with the passed data.
- Syntax: `render_template('template_name.html', variable1=value1, variable2=value2, ...)`
- This is especially useful for making web pages that change based on user input or interactions, such as displaying a user's profile or showing a list of items from a database.

Example:

```
@app.route('/')
def home():
    username = "John"
    return render_template('index.html', username=username)
```

In the example above, the variable `username` is passed to the `index.html` template, where it can be used to display content dynamically, such as:

```
<h1>Welcome, {{ username }}!</h1>
```

2. What is the significance of the templates folder in a Flask project?

In a Flask project, the `templates` folder holds all the HTML files that will be rendered by the Flask application. These HTML files contain the structure of the pages, and Flask will use these templates to dynamically generate the final output sent to the client (user's browser).

Significance:

- **Default Folder for Templates:** By default, Flask looks for HTML templates in a folder named `templates` in the root directory of the project.
- **Separation of Concerns:** The `templates` folder helps to separate the presentation logic (HTML files) from the application logic (Python code). This is an essential part of the Model-View-Controller (MVC) design pattern, where the templates represent the View.
- **Dynamic Rendering:** The templates are used with `render_template()` to generate HTML dynamically by integrating data passed from Flask. This allows Flask to send customized responses to the user, such as a user profile page or a product listing page.
- **Reusability:** Templates can be reused across different parts of the application. For example, you might have a base template that includes a header and footer, and other templates can inherit from this base template to avoid code repetition.
-

3. What is Jinja2, and how does it integrate with Flask?

Jinja2 is a templating engine for Python that is used to render dynamic content within HTML templates. Flask uses Jinja2 by default for rendering HTML templates, enabling the inclusion of dynamic data and control structures in the template files.

Key Features of Jinja2:

- **Template Inheritance:** Jinja2 allows the creation of base templates, from which other templates can inherit common layout structures (such as headers, footers, navigation menus).
- **Variables:** You can pass data from your Flask view functions to templates as variables. These variables can be dynamically inserted into the HTML.

- Control Structures: Jinja2 supports control structures like loops and conditionals, allowing for dynamic content generation based on the passed data.
- Filters: Jinja2 allows you to apply filters to variables, which can modify the data before rendering (e.g., formatting dates, converting text to uppercase).
- Includes: Jinja2 allows including other templates within the current template, making it easy to reuse components like headers, footers, or sidebars.

How Jinja2 integrates with Flask:

- Flask automatically integrates Jinja2 with its template rendering system, so when you use `render_template()`, Flask uses Jinja2 to process the HTML files.
- You can use Jinja2 syntax within the HTML files stored in the templates folder to dynamically display content passed from the Flask view functions.

Example Jinja2 Syntax in a Template:

- Variables: `{{ variable_name }}`
- Conditional Statements: `{% if condition %} ... {% endif %}`
- Loops: `{% for item in items %} ... {% endfor %}`
- Filters: `{{ variable | filter_name }}`

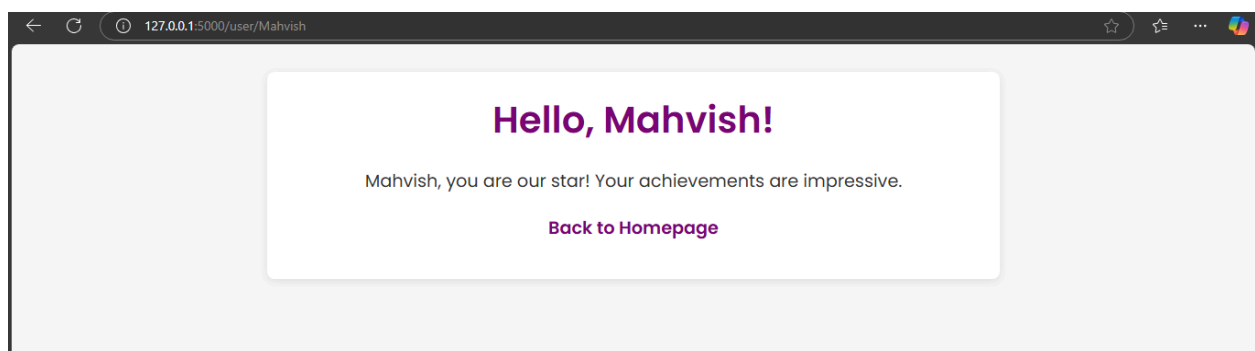
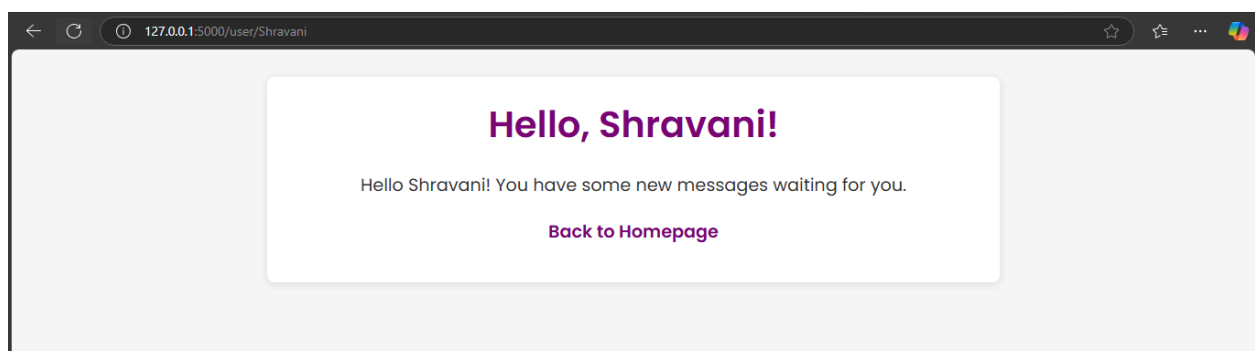
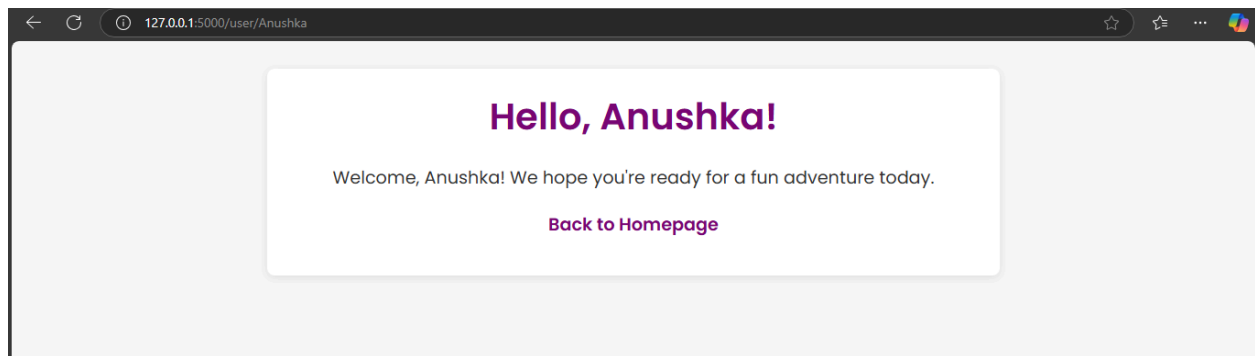
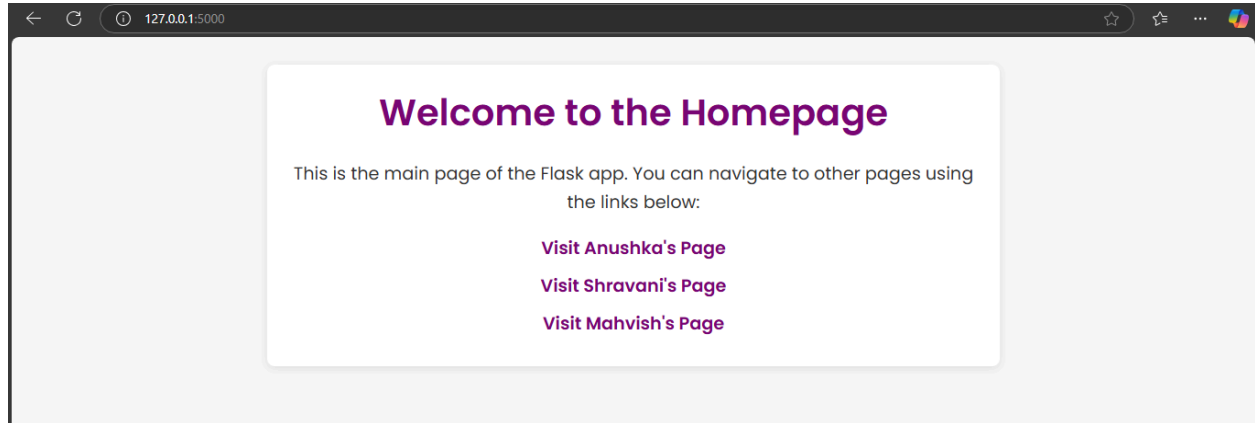
Example:

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>User Page</title>
</head>
<body>
  <h1>Welcome, {{ username }}!</h1>

  {% if username == 'John' %}
    <p>John, you're our star!</p>
  {% elif username == 'Jane' %}
    <p>Hello Jane! You have new notifications.</p>
  {% else %}
    <p>Welcome to your profile, {{ username }}!</p>
  {% endif %}
</body>
</html>
```

OUTPUT :



Github link : https://github.com/Anushka3204/Webx_Exp_5

Conclusion : This Flask application successfully demonstrates template rendering using Jinja2, allowing for dynamic HTML content generation. By incorporating variables and control structures in templates, the app personalizes the user experience based on input. The use of the `render_template()` function and Flask's integration with Jinja2 effectively separates application logic from presentation, enhancing maintainability and reusability.